

Завдання 1. ArcadeGame

Які проблеми є в коді

1. Порушено Open/Closed Principle

Код погано розширюється. Якщо додати нового монстра, доведеться змінювати вже існуючий код:

- додати нове значення в MonsterType,
- додати новий список у GameScene,
- дописати новий case у switch,
- дописати новий цикл у start().

Тобто клас GameScene не закритий для змін.

2. Порушено Single Responsibility Principle

GameScene виконує одразу кілька задач:

- зберігає монстрів,
- створює їх,
- визначає тип монстра,
- запускає їх дії.

Для одного класу це забагато відповідальності.

3. Порушено Dependency Inversion Principle

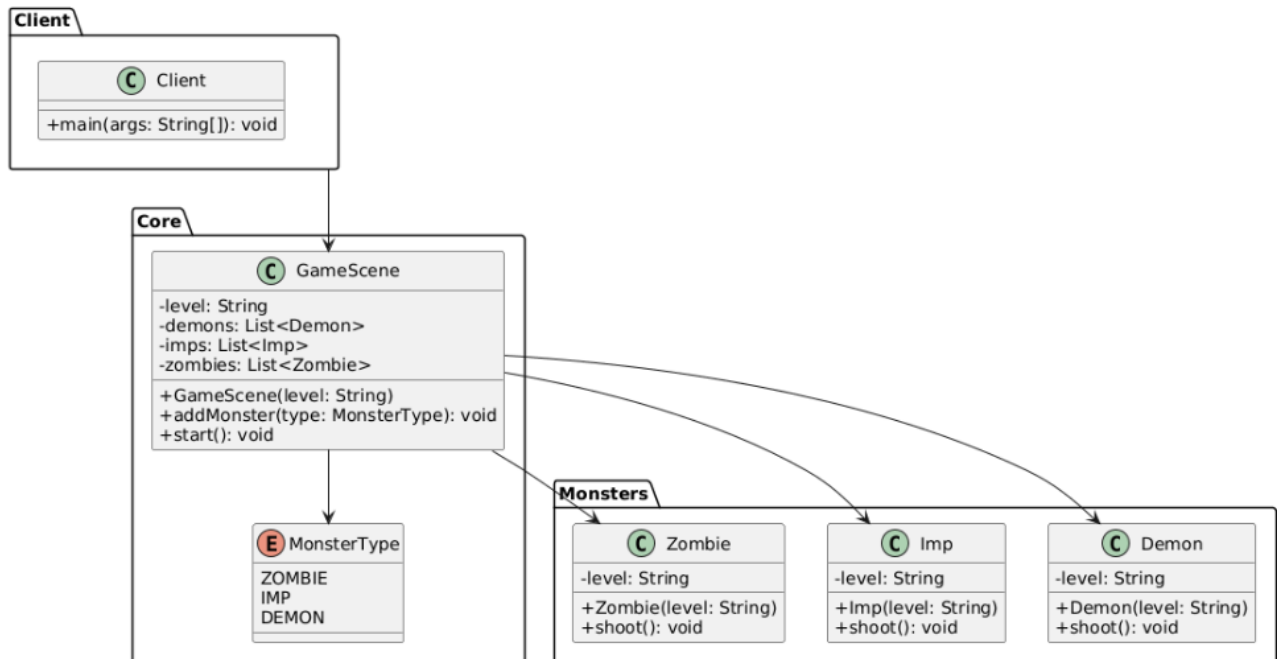
GameScene залежить від конкретних класів Zombie, Imp, Demon, а не від спільної абстракції.

Через це код сильно прив'язаний до конкретних реалізацій.

4. Слабке використання поліморфізму

У всіх монстрів є однакова поведінка — метод shoot(), але спільного інтерфейсу або базового класу немає.

Замість поліморфізму використано enum, switch і окремі списки.



Наскільки легко додавати нові типи монстрів

Новий тип монстра додавати **незручно**.

Наприклад, щоб додати `Cacodemon`, треба:

1. створити клас `Cacodemon`,
2. додати його в `MonsterType`,
3. додати новий список у `GameScene`,
4. додати новий case у `addMonster()`,
5. додати новий цикл у `start()`.

Тобто потрібно змінювати код у кількох місцях. Це незручно і не відповідає хорошему ООП-дизайну.

Які зміни я б зробив у першу чергу

1. Створив би спільний інтерфейс `Monster`

Наприклад:

- `void shoot();`

2. Усі класи монстрів реалізували б цей інтерфейс

- `Zombie implements Monster`
- `Imp implements Monster`
- `Demon implements Monster`

3. У GameScene залишив би один список

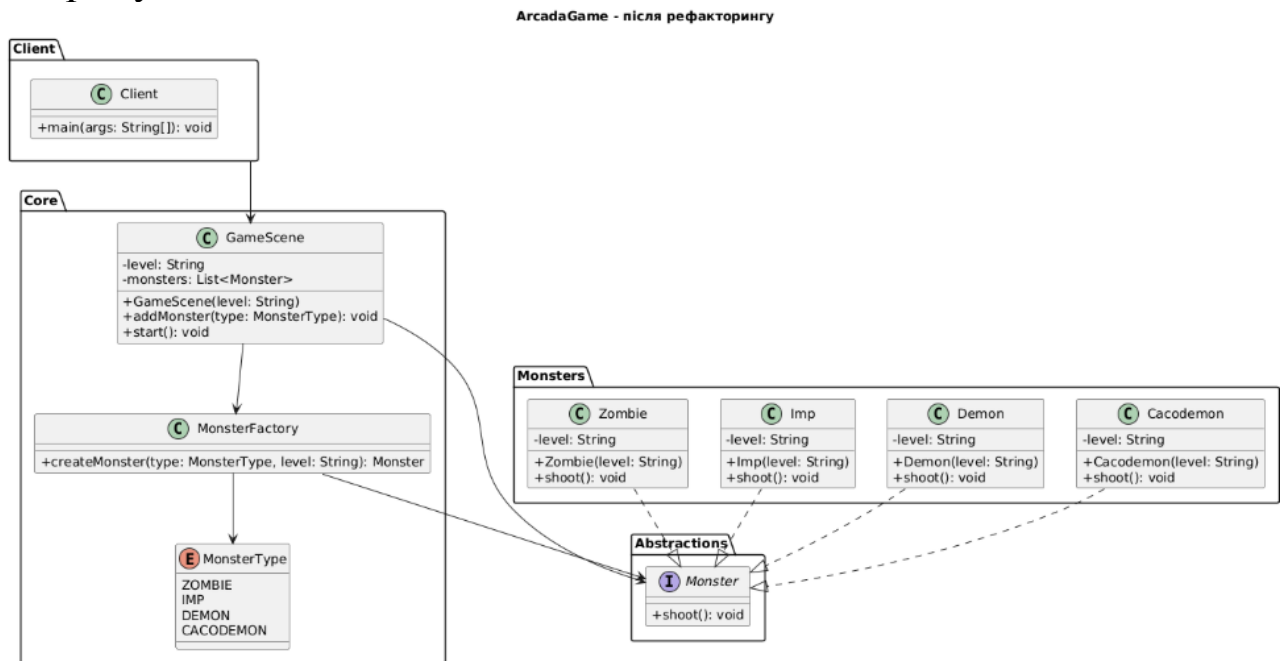
- List<Monster> monsters

4. У start() використовував би один цикл

Для кожного монстра просто викликав би shoot().

5. Створення монстрів виніс би в окрему фабрику

Наприклад, MonsterFactory, щоб GameScene не створював монстрів
напрямую.



Висновок

У поточному вигляді код працює, але має погану масштабованість.
Основні проблеми:

- порушення OCP,
- порушення SRP,
- залежність від конкретних класів,
- відсутність нормального поліморфізму.

Найкраще рішення — ввести спільну абстракцію `Monster`, зберігати всіх монстрів в одному списку і винести їх створення в окрему фабрику.

Завдання 2. DataBase

Які проблеми є в коді

1. Порушено Dependency Inversion Principle

MSDataBaseClient залежить від конкретного класу MSDataBase, а не від абстракції.

Через це, якщо треба буде підключити іншу БД, наприклад PostgresDataBase або MongoDataBase, доведеться змінювати клієнтський код.

2. Порушено Open/Closed Principle

Система погано розширюється.

Якщо з'явиться новий тип бази даних або новий спосіб роботи з нею, код клієнта доведеться змінювати, а не просто додавати нову реалізацію.

3. Порушено Single Responsibility Principle

MSDataBaseClient не тільки працює з базою, а ще й містить логіку вибору операцій через if / else.

Тобто в одному класі змішані:

- робота з БД;
- керування сценарієм виконання.

Це краще розділити.

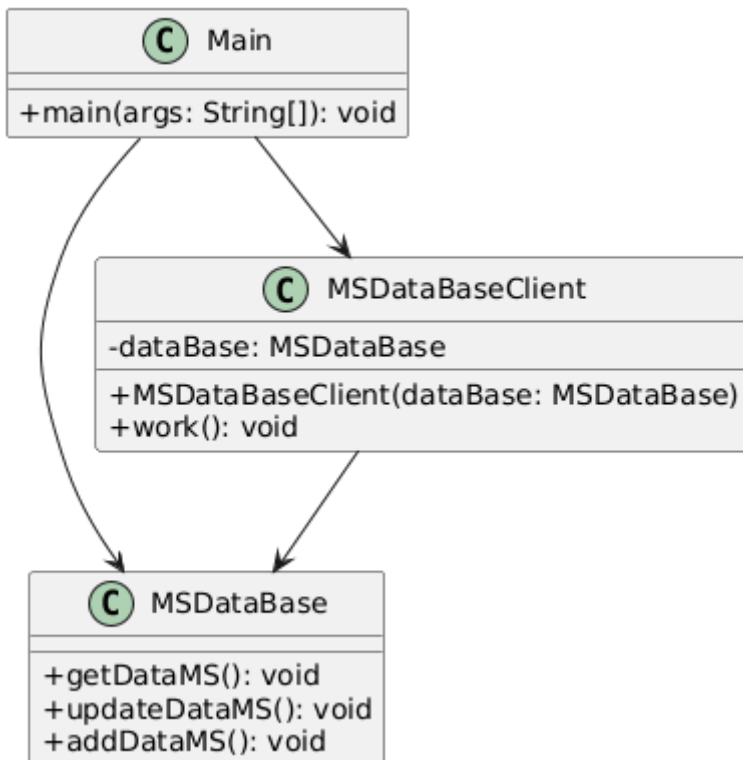
4. Жорстка прив'язка до конкретної реалізації

Назви методів типу getDataMS(), updateDataMS(), addDataMS() теж показують, що код заточений саме під MSDataBase.

Краще мати спільний інтерфейс із загальними методами:

- getData()
- updateData()
- addData()

DataBase - поточний стан



Наскільки легко додати нову базу даних

У поточному вигляді — **не дуже зручно**.

Наприклад, якщо треба додати PostgresDataBase, то доведеться:

1. створити новий клас;
2. змінювати MSDataBaseClient, бо він працює тільки з MSDataBase;
3. можливо змінювати Main.

Тобто клієнт не універсальний. Він не може працювати з різними БД через спільний контракт.

Які зміни я б зробив у першу чергу

1. Створив би інтерфейс DataBase

Наприклад:

- addData()
- updateData()
- getData()

2. MSDataBase реалізував би цей інтерфейс

Тоді клас стане однією з реалізацій БД.

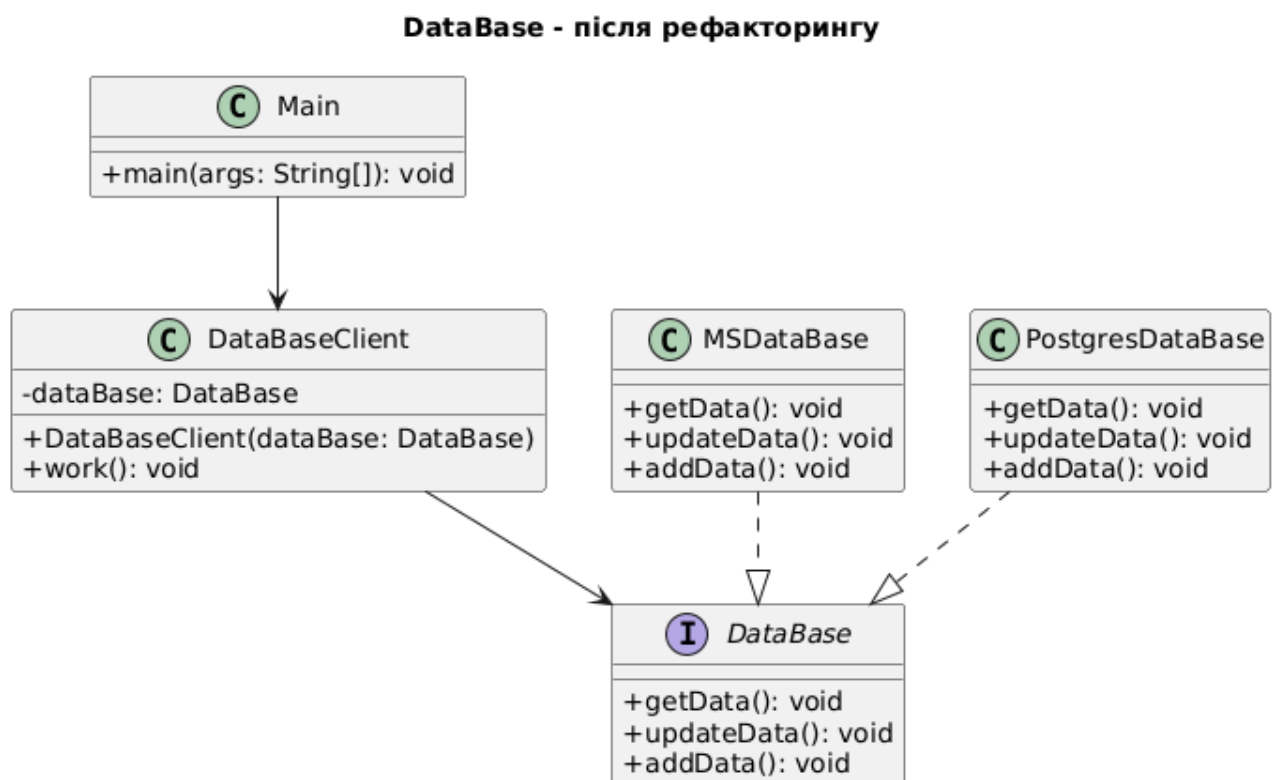
3. MSDataBaseClient залежав би від DataBase, а не від MSDataBase

Тобто в конструктор передавалась би абстракція.

4. Логіку сценарію виконання бажано винести окремо

Метод `work()` зараз виглядає штучно і містить жорстко зашитий алгоритм через `i % 3`.

Для реальної архітектури краще, щоб клієнт просто викликав потрібні методи БД, а не вирішував усе всередині себе.



Висновок

У поточному варіанті код працює, але має жорстку прив'язку до `MSDataBase`.

Нормальний варіант рефакторингу — ввести інтерфейс `DataBase` і змусити клієнт працювати через нього. Тоді можна буде легко підміняти реалізації бази даних без змін у клієнтському коді.

Завдання 3. Vehicles

Які проблеми є в коді

1. Порушено Liskov Substitution Principle (LSP)

Принцип LSP означає, що об'єкти дочірніх класів повинні коректно підставлятися замість базового класу.

У цьому коді Bicycle формально є Vehicle, але поводить себе не так, як очікується від Vehicle.

Якщо десь у програмі працювати з об'єктом типу Vehicle і викликати `drive()`, то для Car усе буде нормально, а для Bicycle програма впаде з винятком.

Тобто Bicycle не можна безпечно використовувати там, де очікується звичайний Vehicle з методом `drive()`.

Це і є порушення LSP.

2. Порушено Interface Segregation Principle (ISP)

Проблема в тому, що базовий клас фактично нав'язує всім нащадкам метод `drive()`, хоча не кожен транспорт має однакову модель руху.

Для автомобіля `drive()` підходить, а для велосипеда — ні.

Отже, інтерфейс або базовий клас вийшов занадто загальним і невдалим.

Правильніше було б не змушувати всі види транспорту підтримувати одну й ту саму операцію.

3. Невдала ієрархія успадкування

Зараз успадкування побудоване за принципом “усі транспортні засоби мають однакову поведінку”, але це не так.

Автомобіль і велосипед — це різні види транспорту з різною логікою руху.

Через це базовий клас Vehicle містить метод, який підходить не всім підкласам.

Це ознака слабкої абстракції.

4. Жорстка прив'язка Car до конкретного двигуна

У класі Car двигун створюється прямо всередині конструктора:

```
this.engine = new GasolineEngine();
```

Це означає, що Car жорстко залежить від конкретного класу GasolineEngine.

Якщо треба буде зробити електромобіль або авто з іншим типом двигуна, код доведеться змінювати.

Тут вже проглядається проблема з гнучкістю і залежністю від конкретної реалізації.

Чому це погано

Такий код працює тільки для дуже простого прикладу, але погано масштабується.

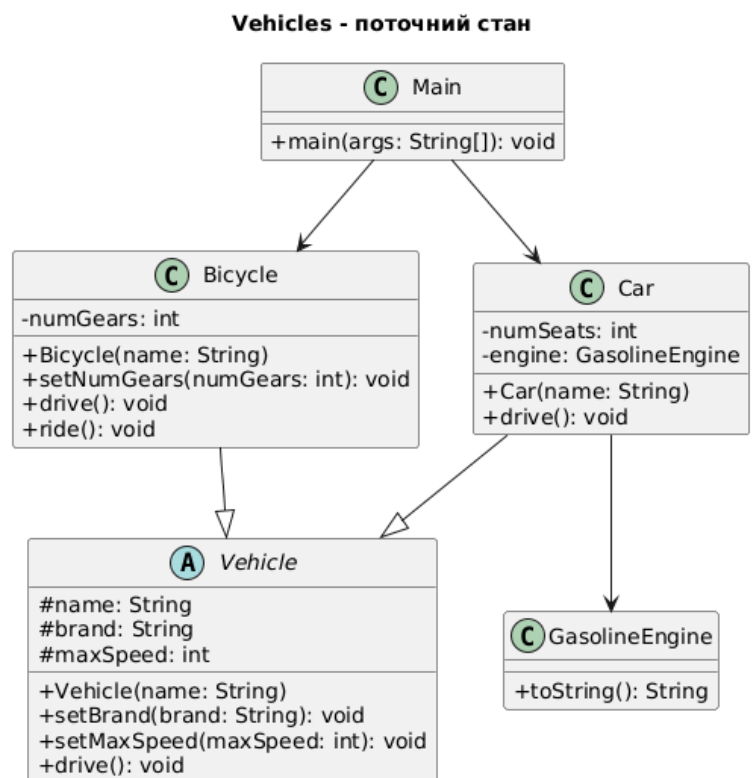
Якщо додати нові типи транспорту, наприклад:

- Motorcycle,
- Scooter,
- Airplane,
- Boat,

то дуже швидко стане зрозуміло, що універсальний метод drive() не підходить усім однаково.

Через це доведеться:

- або кидати винятки в підкласах,
- або створювати порожні / неприродні методи,
- або ламати базову ієрархію.



Як це краще виправити

1. Прибрати універсальний drive() з базового класу Vehicle

Базовий клас повинен містити лише спільні характеристики:

- name
- brand
- maxSpeed

А конкретну поведінку руху треба винести в окремі інтерфейси.

2. Ввести окремі інтерфейси поведінки

Наприклад:

- Drivable — для тих, хто може drive();
- Rideable — для тих, хто може ride().

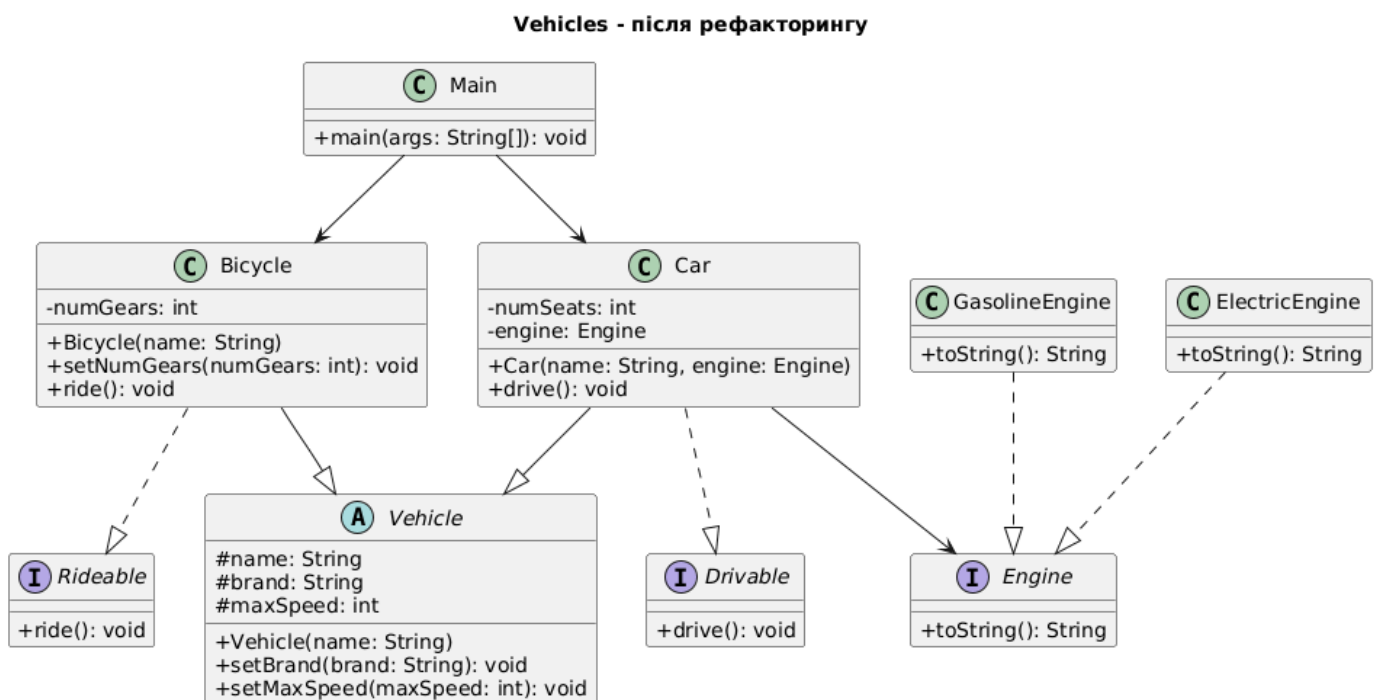
Тоді:

- Car реалізує Drivable;
- Bicycle реалізує Rideable.

3. Винести двигун в абстракцію

Замість конкретного GasolineEngine краще зробити інтерфейс Engine, а вже GasolineEngine буде однією з реалізацій.

Тоді Car залежатиме від абстракції Engine, а не від конкретного бензинового двигуна.



Висновок

У цьому прикладі основна проблема — невдала модель успадкування.

Порушення такі:

- **LSP** — Bicycle не можна нормально підставити замість Vehicle, бо `drive()` для нього непридатний;
- **ISP** — базовий клас нав'язує метод, який підходить не всім;
- також є зайва жорстка залежність Car від GasolineEngine.

Нормальне рішення:

- залишити Vehicle як базовий клас тільки для спільних даних;
- поведінку руху винести в окремі інтерфейси;
- двигун теж винести в абстракцію.